

Strings

```
import android.content.SharedPreferences;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.constraintlayout.widget.ConstraintLayout;
import com.mobilehackinglab.challenge.databinding.ActivityMainBinding;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;
import kotlin.Metadata;
import kotlin.jvm.internal.Intrinsics;

/* compiled from: MainActivity.kt */
@Metadata(d1 = {"\u0000\u0002\n\u0002\u0002\u0002\n\u0002\u0002\u0002\u0002\n\u0002\u0002\u0002\b\u0002\u0002\n\u0002\u0002\u0002\u0002\u0002\u0002\u0002\n\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002\u0002"}, loadedFrom = "classes.dex")
public final class MainActivity extends AppCompatActivity {
    private ActivityMainBinding binding;

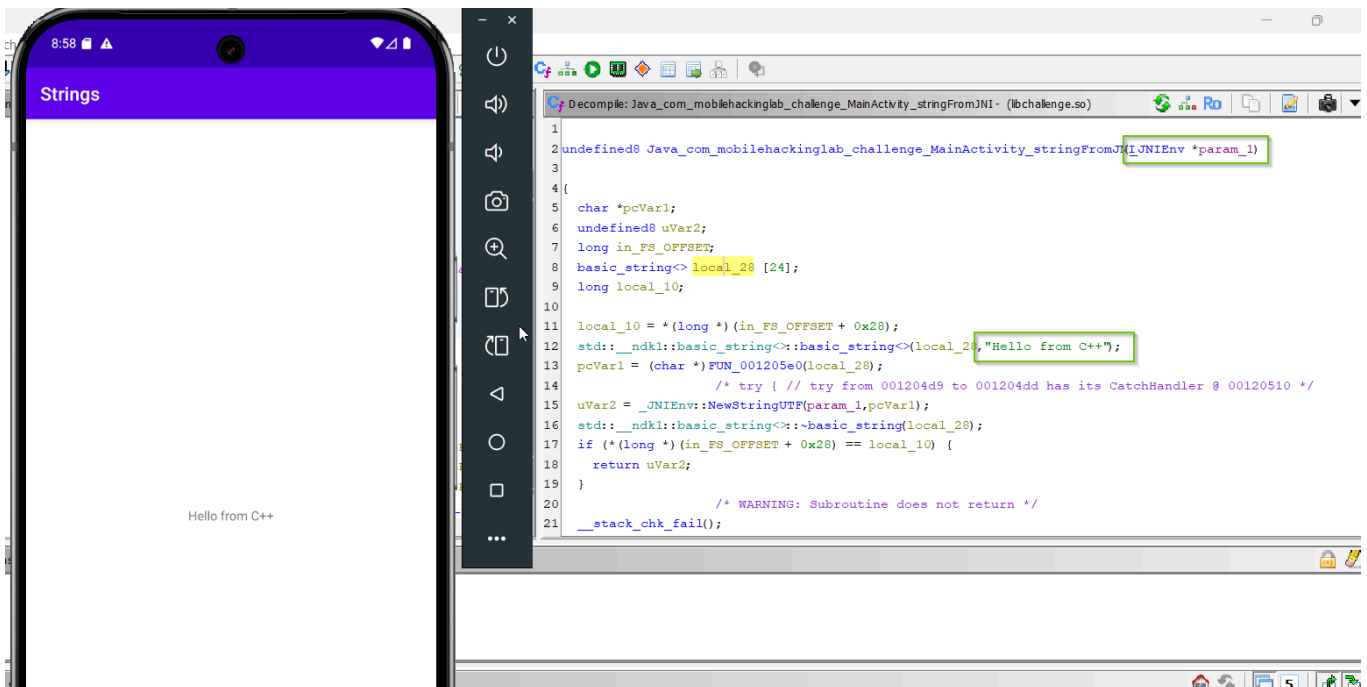
    public final native String stringFromJNI();

    @Override // androidx.fragment.app.FragmentActivity, androidx.activity.ComponentActivity, androidx.core.app.ComponentActivity, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        ActivityMainBinding activityMainBindingInflate = ActivityMainBinding.inflate(getLayoutInflater());
        Intrinsics.checkNotNullExpressionValue(activityMainBindingInflate, "inflate(...)");
        this.binding = activityMainBindingInflate;
        ActivityMainBinding activityMainBinding = null;
        if (activityMainBindingInflate == null) {
            Intrinsics.throwUninitializedPropertyAccessException("binding");
            activityMainBindingInflate = null;
        }
        setContentView(activityMainBindingInflate.getRoot());
        ActivityMainBinding activityMainBinding2 = this.binding;
        if (activityMainBinding2 == null) {
            Intrinsics.throwUninitializedPropertyAccessException("binding");
        } else {
            activityMainBinding = activityMainBinding2;
        }
        activityMainBinding.sampleText.setText(stringFromJNI());

        static {
            System.loadLibrary("challenge");
        }

        public final void KLOW() {
            SharedPreferences sharedPreferences = getSharedPreferences("DAD4", 0);
            SharedPreferences.Editor editor = sharedPreferences.edit();
            Intrinsics.checkNotNullExpressionValue(editor, "edit(...)");
            SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy", Locale.getDefault());
            String cu_d = sdf.format(new Date());
            editor.putString("UUU0133", cu_d);
            editor.apply();
        }
}
```

Starting with reversing the main activity , got my attention the presence of the native function so i reversed the library with ghidra to see its logic



It seems like this function just prints the "Hello from c++" message and nothing interesting going on here as it takes no parameters , so i guess i am moving to the next activity

But the shared preference and time storing seems it has something to do with the flag validation like it can be used as seed or whatever they are doing with it , we will see + another odd thing ,

The main activity never opens the second activity and it is exported

but now lets analyze the next activity

[illegible]

Definitely the flag validation is done here

The screenshot shows a mobile application interface on the left and a decompiler window on the right. The mobile app has a purple header with the text "Strings" and a white body with the text "Hello from C++". The decompiler window shows the decompiled code for the function `Java_com_mobilehackinglab_challenge_Activity2_getflag(JNIEnv *param_1)`. The code includes several function calls and variable assignments, such as `flag_fill()`, `flag13()`, `ZGS(0x2a)`, `tyy(10, 0x14)`, `asds(5)`, `zhs()`, `x1(0x1e, 0xf)`, `ss(8)`, and `flag13()`.

```
private final String cd() {
    SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy", Locale.getDefault());
    String str = sdf.format(new Date());
    Intrinsic.checkNotNullExpressionValue(str, "format(...)");
    Activity2Kt.cu_d = str;
    String str2 = Activity2Kt.cu_d;
    if (str2 != null) {
        return str2;
    }
    Intrinsic.throwUninitializedPropertyAccessException("cu_d");
    return null;
}
```

but we can use frida for sure to bypass this checks

since we already understand the required conditions to call the `get flag` we can simply utilize FRIDA to get these values and then pass them to the activity to get the flag for us

so we need to set the shared preference value from the main activity that the `cd()` function will validate , we don't need to get its value

so we need to set the shared preference value from the main activity that the `cd()` function will validate , we don't need to get its value

```

JS mobilejs > ...
1  Java.perform(function(){
2      Java.choose("com.mobilehackinglab.challenge.MainActivity",{
3          ...    onMatch:function(instance){
4              ...    console.log("Main activity class"+instance);
5              ...    instance.KLOW();
6              ...    },
7          ...    onComplete:function(){
8              ...    };
9      });
10 });

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

        instance.KLOW();
    },
    onComplete:function(){
    });
});
Main activity classcom.mobilehackinglab.challenge.MainActivity@549f502
[Android Emulator 5554::PID::5868 ]->

```

Ln 1, Col 1 (275 selected) Spaces: 4 UTF-8 CRLF

and now we can move on to the next conditions to be met

the activity validate the base64 secret then call the native get flag function

easy to tell what needed just a deep link triggering intent can be sent using adb command or developed app

but what exactly the deep link ?

easy -> it is just base64 encoded encrypted secret key

```

boolean isUIMatching = Intrinsic.areEqual(u_1, cd());
if (isActionView && isUIMatching) {
    Uri uri = getIntent().getData();
    if (uri != null && Intrinsic.areEqual(uri.getScheme(), "mhl") && Intrinsic.areEqual(uri.getHost(), "labs")) {
        String base64Value = uri.getLastPathSegment();
        byte[] decodedValue = Base64.decode(base64Value, 0);
        if (decodedValue != null) {
            String ds = new String(decodedValue, Charsets.UTF_8);
            byte[] bytes = "your_secret_key_1234567890123456".getBytes(Charsets.UTF_8);
            Intrinsic.checkNotNullExpressionValue(bytes, "this as java.lang.String.getBytes(charset)");
            String str = decrypt("AES/CBC/PKCS5Padding", "bqGrDKdQ8zo26HfLrSgvVA==", new SecretKeySpec(bytes, "AES"));
            if (str.equals(ds)) {
                System.loadLibrary("flag");
                String s = getflag();
                Toast.makeText(getApplicationContext(), s, 1).show();
                return;
            } else {
                finishAffinity();
                finish();
                System.exit(0);
                return;
            }
        }
    }
}
finishAffinity();

```

with all needed input for AES algorithm to work we can generate the secret encrypted key to call the function and get the flag

```

public static final String fixedIV = "1234567890123456";

```

So let's do it :

```
key.py
1  from base64 import b64decode
2  from Crypto.Cipher import AES
3
4  key = b"your_secret_key_1234567890123456"
5  cipher_text_b64 = "bqGrDKdQ8zo26Hf1RsGvVA=="
6  iv = b"1234567890123456"
7  cipher = AES.new(key, AES.MODE_CBC, iv)
8  decrypted = cipher.decrypt(b64decode(cipher_text_b64))
9  pad_len = decrypted[-1]
10 print(decrypted[:-pad_len].decode())

I

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

}});
Main activity classcom.mobilehackinglab.challenge.MainActivity@b49f502
[Android Emulator 5554::PID::5868 ]-> exit

Thank you for using Frida!
PS C:\Users\Dell\Downloads\frida> python .\key.py
mhl_secret_1337
PS C:\Users\Dell\Downloads\frida>
```

```
PS C:\Users\Dell\Downloads\frida> adb shell am start -a android.intent.action.VIEW -d "mhl://labs/bwnsx3N1Y3JldF8xM2M3" com.mobilehackinglab.challenge
Starting: Intent { act=android.intent.action.VIEW dat=mhl://labs/... pkg=com.mobilehackinglab.challenge }
PS C:\Users\Dell\Downloads\frida>
```

ok it printed success but doesn't seem to leak the flag in the logs or anywhere

```
Pixel 9 Pro XL (emulator-5554) Android 16, API 36.0  MHL
2025-10-15 21:23:52.610 854-2321 ActivityTaskManager system_server I START u0 {act=android.intent.action.view dat=mhl://labs/... flg=0x10000000 xflg=0x4} with LAUNCH_MULTIPLE from uid 2000
2025-10-15 21:24:29.021 854-1380 ActivityTaskManager system_server I START u0 {act=android.intent.action.VIEW dat=mhl://labs/... flg=0x10000000 xflg=0x4 cmp=com.mobilehackinglab.challenge/...
2025-10-15 21:25:25.508 854-876 ActivityTaskManager system_server I START u0 {act=android.intent.action.VIEW dat=mhl://labs/... flg=0x10000000 xflg=0x4 cmp=com.mobilehackinglab.challenge/...
2025-10-15 21:44:53.246 854-1370 ActivityTaskManager system_server I START u0 {act=android.intent.action.VIEW dat=mhl://labs/... flg=0x10000000 xflg=0x4 pkg=com.mobilehackinglab.challenge
```

So i guess we have to scan the memory to find the flag ourselves

```
Decompile: Java_com_mobilehackinglab_challenge_Activity2_getflag - (libflag.so)

1
2 void Java_com_mobilehackinglab_challenge_Activity2_getflag(JNIEnv *param_1)
3
4 {
5     flag_fill();
6     flag13();
7     ZGS(0x2a);
8     tyy(10, 0x14);
9     asds(5);
10    zhs();
11    x1(0x1e, 0xf);
12    ss(8);
13    flag_fill();
14    flag13();
15    flag13();
16    flag13();
17    ZGS(0x2a);
18    tyy(10, 0x14);
19    asds(5);
20    zhs();
21    dddff();
22    tt();
23    dd();
24    x1(0x1e, 0xf);
25    ss(8);
26    x1(0x1e, 0xf);
27    ss(8);
28    dddff();
29    zhs();
30    _JNIEnv::NewStringUTF(param_1, "Success");
31    return;
32 }
33
```

as the function seems to be building the flag but then directly returns success it doesn't return the flag

so we can scan the memory with the hex format of the beginning of the flag like stated in the description of the challenge

Hints and Rules

- Reverse engineer the application and search for exported activities.
- Understand the code and find a method to invoke the exported activity.
- Utilize Frida for tracing or employ Frida's memory scanning.
- Don't have to spend time on static analysis of the Android library, as the code is obfuscated.
- The flag follows the format "MHL{...}".
- Do not attempt to patch the application.

```

Java.perform(function() {
    console.log("[+] Scanning for MHL{ pattern in memory...");

    Process.enumerateRanges('r--').forEach(function(range) {
        if (range.size < 0x10000000) { // Reasonable size
            Memory.scan(range.base, range.size, "4D 48 4C 7B", { // MHL{ in
hex
                onMatch: function(address, size) {
                    console.log('[+] Pattern found at: ' + address);
                    try {
                        // Read potential flag string
                        var flag = address.readCString();
                        if (flag && flag.includes('}')) {
                            console.log('[FOUND FLAG] ' + flag);
                            send(flag);
                        }
                    } catch(e) {
                        console.log('[!] Error reading string: ' + e);
                    }
                },
                onError: function(reason) {
                    // console.log('[!] Scan error: ' + reason);
                },
                onComplete: function() {
                    // console.log('[+] Scan completed for range: ' +
range.base);
                }
            });
        }
    });
});
});
});

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

});
[+] Scanning for MHL{ pattern in memory...
[Android Emulator 5554::PID::5868 ]-> [i] Pattern found at: 0x75ea489ebfc0
[FOUND FLAG] MHL{IN_THE_MEMORY}
message: {'type': 'send', 'payload': 'MHL{IN_THE_MEMORY}'} data: None

```